

1. Write a C program to store n elements in an array using dynamic memory allocation and find largest number.

code:

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
    int n, i, largest;
    int *a;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    a = (int *)malloc(n * sizeof(int));
    printf("Enter %d elements:\n", n);
    for (i = 0; i < n; i++)
    {
        scanf("%d", &a[i]);
    }
    largest = a[0];
    for (i = 1; i < n; i++)
    {
        if (a[i] > largest)
            largest = a[i];
    }
    printf("Largest number = %d", largest);
    free(a);
    return 0;
}
```

output:

```
Enter number of elements: r    5
Enter 5 elements:
10 20 30 40 50
Largest number = 50
```

2. Print an array in reverse order using pointers

code:

```
#include <stdio.h>
int main()
{
    int a[5], i;
    int *p;
    printf("Enter 5 elements:\n");
    for (i = 0; i < 5; i++)
    {
        scanf("%d", &a[i]);
    }
}
```

```

    p = &a[4];
    printf("Array in reverse order:\n");
    for (i = 4; i >= 0; i--)
    {
        printf("%d ", *p);
        p--;
    }
    return 0;
}

```

output:

```

Enter 5 elements:
10 20 30 40 50
Array in reverse order:
50 40 30 20 10

```

3. Write a C program to store information of Students using structure. Sort them by Name.

code:

```

#include <stdio.h>
#include <string.h>
struct Student
{
    char name[50];
    int age;
    float mark;
};
int main()
{
    struct Student s[5], temp;
    int i, j;
    printf("Enter details of 5 students:\n");
    for (i = 0; i < 5; i++)
    {
        printf("\nEnter name: ");
        scanf("%s", s[i].name);
        printf("Enter age: ");
        scanf("%d", &s[i].age);
        printf("Enter mark: ");
        scanf("%f", &s[i].mark);
    }
    for (i = 0; i < 4; i++)
    {
        for (j = i + 1; j < 5; j++)
        {
            if (strcmp(s[i].name, s[j].name) > 0)
            {
                temp = s[i];
                s[i] = s[j];
                s[j] = temp;
            }
        }
    }
}

```

```

        }
    }
    printf("\nStudents sorted by Name:\n");
    for (i = 0; i < 5; i++)
    {
        printf("%s %d %.2f\n",
               s[i].name, s[i].age, s[i].mark);
    }
    return 0;
}

```

output:

Enter details of 5 students:

Enter name: nittin
Enter age: 19
Enter mark: 95

Enter name: moujin
Enter age: 18
Enter mark: 90

Enter name: shernic
Enter age: 17
Enter mark: 5 80

Enter name: pratheeson
Enter age: 188
Enter mark: 81

Enter name: kabistan
Enter age: 19
Enter mark: 79

Students sorted by Name:
kabistan 19 79.00
moujin 18 90.00
nittin 19 95.00
pratheeson 18 81.00
shernic 17 80.00

4. Write a C program to add 'n' distances given in inch-feet system using structure.

code:

```

#include <stdio.h>
struct Distance
{
    int feet;
    int inch;
};
int main()

```

```

{
    struct Distance d[10], sum = {0, 0};
    int n, i;
    printf("Enter number of distances: ");
    scanf("%d", &n);
    printf("Enter distances (feet inch):\n");
    for (i = 0; i < n; i++)
    {
        scanf("%d %d", &d[i].feet, &d[i].inch);
        sum.feet += d[i].feet;
        sum.inch += d[i].inch;
    }
    sum.feet += sum.inch / 12;
    sum.inch = sum.inch % 12;
    printf("Total distance = %d feet %d inches",
        sum.feet, sum.inch);
    return 0;
}

```

output:

```

Enter number of distances: 3
Enter distances (feet inch):
5 7
8 4
3 9
Total distance = 17 feet 8 inches

```

5. Write a C program to add two complex numbers by passing structure to a function.

code:

```

#include <stdio.h>
struct Complex
{
    float real;
    float imag;
};
struct Complex add(struct Complex c1, struct Complex c2)
{
    struct Complex result;
    result.real = c1.real + c2.real;
    result.imag = c1.imag + c2.imag;
    return result;
}
int main()
{
    struct Complex c1, c2, result;
    printf("Enter first complex number:\n");
    printf("Enter real part: ");
    scanf("%f", &c1.real);
    printf("Enter imaginary part: ");
    scanf("%f", &c1.imag);
}

```

```

    printf("Enter second complex number:\n");
    printf("Enter real part: ");
    scanf("%f", &c2.real);
    printf("Enter imaginary part: ");
    scanf("%f", &c2.imag);
    result = add(c1, c2);
    printf("Sum = %.2f + %.2fi",
    result.real, result.imag);
    return 0;
}

```

output:

```

Enter first complex number:
Enter real part: 5
Enter imaginary part: 8
Enter second complex number:
Enter real part: 4
Enter imaginary part: 6
Sum = 9.00 + 14.00i

```

6. Write a C program to maintain inventory management system.

code:

```

#include <stdio.h>
struct Product
{
    int id;
    char name[30];
    int quantity;
    float price;
};
int main()
{
    struct Product p[10];
    int n, i;
    printf("Enter number of products: ");
    scanf("%d", &n);
    for (i = 0; i < n; i++)
    {
        printf("\nEnter product ID: ");
        scanf("%d", &p[i].id);
        printf("Enter product name: ");
        scanf("%s", p[i].name);
        printf("Enter quantity: ");
        scanf("%d", &p[i].quantity);
        printf("Enter price: ");
        scanf("%f", &p[i].price);
    }
    printf("\n--- Inventory Details ---\n");
    for (i = 0; i < n; i++)
    {
        printf("ID: %d\n", p[i].id);
        printf("Name: %s\n", p[i].name);
    }
}

```

```

        printf("Quantity: %d\n", p[i].quantity);
        printf("Price: %.2f\n\n", p[i].price);
    }
    return 0;
}

```

output:

Enter number of products: 2

Enter product ID: 76
 Enter product name: pen
 Enter quantity: 25
 Enter price: 100

Enter product ID: 78 0 4
 Enter product name: iphone
 Enter quantity: 1
 Enter price: 100000

--- Inventory Details ---

ID: 76
 Name: pen
 Quantity: 25
 Price: 100.00

ID: 74
 Name: iphone
 Quantity: 1
 Price: 100000.00

7. Write a C program for decimal to binary, octal and hexadecimal conversion.

code:

```

#include <stdio.h>
int main()
{
    int n, temp, i;
    int binary[32];
    printf("Enter a decimal number: ");
    scanf("%d", &n);
    temp = n;
    i = 0;
    while (temp > 0)
    {
        binary[i] = temp % 2;
        temp = temp / 2;
        i++;
    }
    printf("Binary = ");
    for (i = i - 1; i >= 0; i--)
        printf("%d", binary[i]);
}

```

```

    printf("\nOctal = %o", n);
    printf("\nHexadecimal = %X", n);
    return 0;
}

```

output:

```

Enter a decimal number: 25
Binary = 11001
Octal = 31
Hexadecimal = 19

```

8. Write a C program to calculate difference between two time period.

code:

```

#include <stdio.h>
struct Time
{
    int hours;
    int minutes;
    int seconds;
};
int main()
{
    struct Time t1, t2, diff;
    printf("Enter first time period (hours minutes seconds): ");
    scanf("%d %d %d", &t1.hours, &t1.minutes, &t1.seconds);
    printf("Enter second time period (hours minutes seconds): ");
    scanf("%d %d %d", &t2.hours, &t2.minutes, &t2.seconds);
    if (t2.seconds < t1.seconds)
    {
        t2.seconds += 60;
        t2.minutes--;
    }
    if (t2.minutes < t1.minutes)
    {
        t2.minutes += 60;
        t2.hours--;
    }
    diff.seconds = t2.seconds - t1.seconds;
    diff.minutes = t2.minutes - t1.minutes;
    diff.hours = t2.hours - t1.hours;
    printf("Time Difference = %d hours %d minutes %d seconds",
        diff.hours, diff.minutes, diff.seconds);
    return 0;
}

```

output:

```

Enter first time period (hours minutes seconds): 08 08 07
Enter second time period (hours minutes seconds): 14 01 08
Time Difference = 5 hours 53 minutes 1 seconds

```

9. Write a C program to maintain employee management system using structures.

code:

```
#include <stdio.h>
struct Employee
{
    int id;
    char name[30];
    float salary;
};
int main()
{
    struct Employee e[10];
    int n, i;
    printf("Enter number of employees: ");
    scanf("%d", &n);
    for (i = 0; i < n; i++)
    {
        printf("\nEnter employee ID: ");
        scanf("%d", &e[i].id);
        printf("Enter employee name: ");
        scanf("%s", e[i].name);
        printf("Enter salary: ");
        scanf("%f", &e[i].salary);
    }
    printf("\n--- Employee Details ---\n");
    for (i = 0; i < n; i++)
    {
        printf("ID: %d\n", e[i].id);
        printf("Name: %s\n", e[i].name);
        printf("Salary: %.2f\n\n", e[i].salary);
    }
    return 0;
}
```

output:

Enter number of employees: 2

Enter employee ID: 100

Enter employee name: nittin

Enter salary: 10000000

Enter employee ID: 101

Enter employee name: moujin

Enter salary: 100000

--- Employee Details ---

ID: 100

Name: nittin

Salary: 10000000.00

ID: 101
Name: moujin
Salary: 100000.00

10. Write a C program to calculate area and perimeter of a rectangle and triangle using function pointers.

code:

```
#include <stdio.h>
float rectangleArea(float l, float b)
{
    return l * b;
}
float rectanglePerimeter(float l, float b)
{
    return 2 * (l + b);
}
float triangleArea(float b, float h)
{
    return 0.5 * b * h;
}
float trianglePerimeter(float a, float b, float c)
{
    return a + b + c;
}
int main()
{
    float l, b, h, a, c;
    float (*area)(float, float);
    float (*perimeter)(float, float);
    printf("Enter length and breadth of rectangle: ");
    scanf("%f %f", &l, &b);
    area = rectangleArea;
    perimeter = rectanglePerimeter;
    printf("Rectangle Area = %.2f\n", area(l, b));
    printf("Rectangle Perimeter = %.2f\n", perimeter(l, b));
    printf("\nEnter base and height of triangle: ");
    scanf("%f %f", &b, &h);
    printf("Enter other two sides of triangle: ");
    scanf("%f %f", &a, &c);
    area = triangleArea;
    printf("Triangle Area = %.2f\n", area(b, h));
    printf("Triangle Perimeter = %.2f",
           trianglePerimeter(a, b, c));
    return 0;
}
```

output:

Enter length and breadth of rectangle: 5 10
Rectangle Area = 50.00
Rectangle Perimeter = 30.00

```
Enter base and height of triangle: 56    6
Enter other two sides of triangle: 7 8
Triangle Area = 15.00
Triangle Perimeter = 20.00
```